

Группа	Z3342	К работе допущен	_____
Студент	Погорелко В. А.	Работа выполнена	_____
Преподаватель	Маслов И. Д.	Отчёт принят	_____

Рабочий протокол и отчёт по лабораторной работе №1

UNIX знакомство: useradd, nano, chmod, docker, GIT, CI, CD

1. Цель работы

Познакомить студента с основами администрирования программных комплексов в ОС семейства UNIX, продемонстрировать особенности виртуализации и контейнеризации, продемонстрировать преимущества использования систем контроля версий (на примере GIT)

2. Задачи

1. (ОС) Работа в ОС, использование файловой системы, прав доступа, исполнение файлов
2. (КОНТЕЙНЕР) docker build / run / ps / images
3. (GIT) GitHub / GitLab, в котором будут содержаться все выполненные ЛР
4. (SAVE) Всё, что было сделано в шагах 1-3, сохранить в репозиторий (+ отчет по данной ЛР в папку doc). Фиксацию ревизий производить строго через ветку dev. С помощью скриптов накатить ревизии на stg и на prd.

3. Решение

1. Работа в ОС, использование файловой системы, прав доступа, исполнение файлов

1.1. В папке /USR/LOCAL/ создать 2 директории: folder_max, folder_min

Запросы:

```
boris@Boris:~$ cd /.
boris@Boris:/$ cd /usr/local
boris@Boris:/usr/local$ sudo mkdir folder_max
[sudo] password for boris: #password
boris@Boris:/usr/local$ sudo mkdir folder_min
```

Результат:

```
boris@Boris:/usr/local$ ls -l
total 40
drwxr-xr-x 2 root root 4096 Feb 10 03:54 bin
drwxr-xr-x 2 root root 4096 Feb 10 03:54 etc
drwxr-xr-x 2 root root 4096 Mar 28 11:08 folder_max #created
drwxr-xr-x 2 root root 4096 Mar 28 11:08 folder_min #created
drwxr-xr-x 2 root root 4096 Feb 10 03:54 games
drwxr-xr-x 2 root root 4096 Feb 10 03:54 include
drwxr-xr-x 3 root root 4096 Feb 10 03:54 lib
...
```

1.2. Создание 2-х групп пользователей: group_max, group_min

Запросы:

```
boris@Boris:/usr/local$ sudo groupadd group_max
[sudo] password for boris: #password
boris@Boris:/usr/local$ sudo groupadd group_min
```

Результат:

```
boris@Boris:/etc$ cat /etc/group
...
admin:x:106:
netdev:x:107:
boris:x:1000:
group_max:x:1001: #created
group_min:x:1002: #created
```

1.3. Создать 2-х пользователей: user_max_1, user_min_1

Запросы:

```
boris@Boris:/usr/local$ sudo useradd user_max_1
boris@Boris:/usr/local$ sudo useradd user_min_1
```

Результат:

```
boris@Boris:/etc$ id user_max_1
uid=1001(user_max_1) gid=1003(user_max_1) /
/ groups=1003(user_max_1) #created
boris@Boris:/etc$ id user_min_1
uid=1002(user_min_1) gid=1004(user_min_1) /
groups=1004(user_min_1) #created
```

1.4. Для пользователей из группы *_max дать полный доступ на директории *_max и *_min. Для пользователей группы *_min дать полный доступ только на директорию *_min

Сначала раскидаем пользователей по соответствующим группам:

Запросы:

```
boris@Boris:/ $ sudo usermod -aG group_max user_max_1
boris@Boris:/ $ sudo usermod -aG group_min user_min_1
```

Результат:

```
boris@Boris:/ $ groups user_max_1
user_max_1 : user_max_1 group_max #appended
boris@Boris:/ $ groups user_min_1
user_min_1 : user_min_1 group_min #appended
```

Стандартного способа дать папке две группы-владельца нельзя, однако можно установить пакет ACL, который предоставит такую возможность. Сделать это можно следующей командой

Запросы:

```
boris@Boris:/ $ sudo apt install acl
boris@Boris:/ $ sudo setfacl -m g:group_min:rwx \
/usr/local/folder_min
boris@Boris:/ $ sudo setfacl -m g:group_max:rwx \
/usr/local/folder_min
boris@Boris:/ $ sudo setfacl -m g:group_max:rwx \
/usr/local/folder_max
```

Результат:

```
boris@Boris:/$ getfacl folder_max
# file: folder_max
# owner: root
# group: group_max
user::rwx
group::rwx
group:group_max:rwx
mask::rwx
other:---

boris@Boris:/$ getfacl folder_min
# file: folder_max
# owner: root
# group: group_max
user::rwx
group::rwx
group:group_max:rwx
group:group_min:rwx
mask::rwx
other:---
```

1.5. Создать и исполнить (пользователем из той же категории) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в текущей директории

Сначала перейдем в режим использования пользователем `user_max_1`

Запрос:

```
boris@Boris:/usr/local$ sudo -u user_max_1 bash
```

Результат:

```
user_max_1@Boris:/usr/local$
```

Таким запросом мы вошли в ОС требуемым пользователем. Далее по заданию:

Запрос:

```
user_max_1@Boris:/usr/local/folder_max$ nano log_date.sh
user_max_1@Boris:/usr/local/folder_max$ chmod +x log_date.sh
user_max_1@Boris:/usr/local/folder_max$ ./log_date.sh
```

В файл `log_date.sh` должен идти следующий текст:

```
" Inside log_date.sh "  
#!/bin/bash  
data >> output.log
```

Результат:

```
user_max_1@Boris:/usr/local/folder_max$ cat output.log  
Fri Apr 3 19:25:06 MSK 2026 # success
```

1.6. Создать и исполнить (пользователем из той же категории) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в директории `*_min`

Запросы:

```
user_max_1@Boris:/usr/local/folder_max$ nano log_to_min.sh  
user_max_1@Boris:/usr/local/folder_max$ chmod +x log_to_min.sh
```

```
" Inside log_to_min.sh "  
#!/bin/bash  
date >> /usr/local/folder_min/output.log
```

Результаты:

```
user_max_1@Boris:/usr/local/folder_max$ ./log_to_min.sh  
user_max_1@Boris:/usr/local/folder_max$ cd \  
/usr/local/folder_min  
user_max_1@Boris:/usr/local/folder_min$ cat output.log  
Fri Apr 3 19:33:50 MSK 2026 # success
```

1.7. Исполнить (пользователем `*_min`) скрипт в директории `folder_max`, который пишет текущую дату/время в файл `output.log` в директории `*_min`

Запросы:

```
user_max_1@Boris:/usr/local/folder_max$ exit  
boris@Boris:/usr/local$ sudo -u user_min_1 bash  
user_min_1@Boris:/usr/local$ \  
/usr/local/folder_max/log_to_min.sh
```

Результаты:

```
bash: /usr/local/folder_max/log_to_min.sh: Permission denied
user_min_1@Boris:/usr/local$ cd /usr/local/folder_min
user_min_1@Boris:/usr/local/folder_min$ cat output.log
Fri Apr 3 19:33:50 MSK 2026 # failed
```

Таким образом, прочитать уже существующий файл в директории можно, но исполнить файл в директории, прав доступа к которой нет, нельзя.

1.8. Создать и исполнить (пользователем из той же категории) скрипт в директории `folder_min`, который пишет текущую дату/время в файл `output.log` в директории `*_max`

Запросы:

```
user_min_1@Boris:/usr/local/folder_min$ nano log_to_max.sh
user_min_1@Boris:/usr/local/folder_min$ chmod +x log_to_max.sh
```

```
" Inside log_to_max.sh "
#!/bin/bash
date >> /usr/local/folder_max/output.log
```

Результаты:

```
user_min_1@Boris:/usr/local/folder_min$ ./log_to_max.sh
bash: ./log_to_max.sh: Permission denied
user_min_1@Boris:/usr/local/folder_min$ exit
```

1.9. Вывести перечень прав доступа у папок *_min/ *_max, а также у всего содержимого внутри

Запросы и результаты:

```
user_max_1@Boris:/usr/local$ ls -l folder_max
total 12
-rwxrwxr-x 1 user_max_1 user_max_1 31 Apr  3 19:24 log_date.sh
-rwxrwxr-x 1 user_max_1 user_max_1 54 Apr
3 19:32 log_to_min.sh
-rw-rw-r-- 1 user_max_1 user_max_1 29 Apr  3 19:25 output.log

user_max_1@Boris:/usr/local$ ls -l folder_min
total 8
-rwxrwxr-x 1 user_min_1 user_min_1 55 Apr
3 19:38 log_to_max.sh
-rw-rw-r-- 1 user_max_1 user_max_1 29 Apr  3 19:33 output.log
```

2. docker build / run / ps / images

2.1. Создать скрипт, который пишет текущую дату/время в файл output.log в текущей директории

Для начала, выделим отдельную директорию в папке

Запросы:

```
boris@Boris:/usr/local/$ sudo mkdir docker_lab
boris@Boris:/usr/local/$ sudo nano log.sh
```

```
" Inside log.sh "
#!/bin/bash
date >> output.log
```

2.2. Собрать образ со скриптами выше и с пакетом nano (docker build)

Запросы:

```
boris@Boris:/usr/local/$ sudo nano Dockerfile
```

```
" Inside Dockerfile "  
GNU nano 7.2  
Dockerfile  
FROM ubuntu:22.04  
RUN apt-get update && apt-get install -y nano  
WORKDIR /app  
COPY log.sh /app/log.sh  
RUN chmod +x /app/log.sh  
CMD ["/bin/bash"]
```

Далее идет непосредственно сборка контейнера

Запросы:

```
boris@Boris:/usr/local/$ docker build -t my_lab_scripts .
```

Результаты:

```
[+] Building 25.3s (11/11) FINISHED  
docker:default  
=> [internal] load build definition from Dockerfile  
0.0s  
=> => transferring dockerfile: 183B  
0.0s  
=> [internal] load metadata for docker.io/library/ubuntu:22.04  
1.8s  
=> [auth] library/ubuntu:pull token for registry-1.docker.io  
0.0s  
=> [internal] load .dockerignore  
0.0s  
=> => transferring context: 2B  
0.0s  
=> [1/5] FROM ... # lot of text  
=> => transferring context: 64B  
0.0s  
=> [2/5] RUN apt-get update && apt-get install -y nano  
16.8s  
=> [3/5] WORKDIR /app  
0.0s  
=> [4/5] COPY log.sh /app/log.sh  
0.0s  
=> [5/5] RUN chmod +x /app/log.sh  
0.2s  
... # lot of text
```


2.3. Запустить образ (docker run)

Запрос:

```
boris@Boris:/usr/local/docker_lab$ docker run -it my_lab_scripts
```

После запуска мы попадаем внутрь контейнера, сделаем запросы внутри него

Результат:

```
root@e3d8a82d773f:/app# ls -la
total 12
drwxr-xr-x 1 root root 4096 Apr  3 19:24 .
drwxr-xr-x 1 root root 4096 Apr  3 19:28 ..
-rwxr-xr-x 1 root root   31 Apr  3 18:55 log.sh
```

2.4. Выполнить скрипт, который подложили при сборке образа

Запрос:

```
root@e3d8a82d773f:/app# ./log.sh
```

Результат:

```
root@e3d8a82d773f:/app# ls -l
total 8
-rwxr-xr-x 1 root root 31 Apr  3 18:55 log.sh
-rw-r--r-- 1 root root 29 Apr  3 19:28 output.log
root@e3d8a82d773f:/app# cat output.log
Fri Apr  3 19:28:49 UTC 2026
```

2.5. Вывести список пользователей в собранном образе

Запрос:

```
root@c452e9d342ab:/app# cat /etc/passw
```

Результат:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System \
(admin):/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
```

3. GitHub / GitLab, в котором будут содержаться все выполненные ЛР

3.1. Создать репозиторий в GitHub или GitLab

Сначала создадим папку, которая будет нашим локальным репозиторием. На гитхабе у меня уже есть аккаунт, к созданному на нем репозиторию я буду подключать локальную папку.

Запросы:

```
boris@Boris:/usr/local$ sudo mkdir cpp_linux_labs
boris@Boris:/usr/local$ cd cpp_linux_labs
boris@Boris:/usr/local/cpp_linux_labs$ sudo \
chown -R boris:boris /usr/local/cpp_linux_labs
boris@Boris:/usr/local/cpp_linux_labs$ git init
boris@Boris:/usr/local/cpp_linux_labs$ git remote \
add origin https://github.com/sevsvs/cpp_linux_labs.git
```

Результат:

```
boris@Boris:/usr/local/cpp_linux_labs$ git remote -v
origin  https://github.com/sevsvs/cpp_linux_labs.git (fetch)
origin  https://github.com/sevsvs/cpp_linux_labs.git (push)
```

3.2. Создать структуру репозитория

Запросы:

```
boris@Boris:/usr/local/cpp_linux_labs$
```

```
mkdir -p lab_01/build
mkdir -p lab_01/src
mkdir -p lab_01/doc
mkdir -p lab_01/cmake
```

```
mkdir -p lab_02/build
mkdir -p lab_02/src
mkdir -p lab_02/doc
mkdir -p lab_02/cmake
```

Результат:

```
boris@Boris:/usr/local/cpp_linux_labs$ ls -l
total 8
drwxr-xr-x 6 boris boris 4096 Apr  3 23:19 lab_01
drwxr-xr-x 6 boris boris 4096 Apr  3 23:20 lab_02

boris@Boris:/usr/local/cpp_linux_labs$ cd lab_01
boris@Boris:/usr/local/cpp_linux_labs/lab_01$ ls -l
total 16
drwxr-xr-x 2 boris boris 4096 Apr  3 23:19 build
drwxr-xr-x 2 boris boris 4096 Apr  3 23:19 cmake
drwxr-xr-x 2 boris boris 4096 Apr  3 23:19 doc
drwxr-xr-x 2 boris boris 4096 Apr  3 23:19 src
```

Также, в каждый файл я засунул пустышки .gitkeep, они необязательны, но желательны для корректного функционирования пустых папок. Далее добавим измененные файлы в буфер, зафиксируем коммит изменений и запустим изменения в main ветку.

Запросы:

```
boris@Boris:/usr/local/cpp_linux_labs$ git add .
boris@Boris:/usr/local/cpp_linux_labs$ git \
commit -m "init commit"
[master (root-commit) a556adc] init commit
8 files changed, 0 insertions(+), 0 deletions(-)
create mode 100644 lab_01/build/.gitkeep
create mode 100644 lab_01/cmake/.gitkeep
create mode 100644 lab_01/doc/.gitkeep
create mode 100644 lab_01/src/.gitkeep
create mode 100644 lab_02/build/.gitkeep
create mode 100644 lab_02/cmake/.gitkeep
create mode 100644 lab_02/doc/.gitkeep
create mode 100644 lab_02/src/.gitkeep
boris@Boris:/usr/local/cpp_linux_labs$ git branch -M main
boris@Boris:/usr/local/cpp_linux_labs$ git push -u origin main
```

Далее с помощью токена я подтвердил отправку изменений на Github.

3.3. Создать ветки dev / stg / prd, удалить ранее существующие ветки удаленно и локально

Создание веток делается также стандартными командами.

Запросы:

```
boris@Boris:/usr/local/cpp_linux_labs$ git checkout -b dev
Switched to a new branch 'dev'
boris@Boris:/usr/local/cpp_linux_labs$ git checkout -b stg
Switched to a new branch 'stg'
boris@Boris:/usr/local/cpp_linux_labs$ git checkout -b prd
Switched to a new branch 'prd'
```

Аналогично пушим эти изменения на Github.

Запросы:

```
boris@Boris:/usr/local/cpp_linux_labs$ git push -u origin dev
... # succesful
boris@Boris:/usr/local/cpp_linux_labs$ git push -u origin stg
... # succesful
boris@Boris:/usr/local/cpp_linux_labs$ git push -u origin prd
... # succesful
```

Результат:

```
boris@Boris:/usr/local/cpp_linux_labs$ git branch
* dev
  main
  prd
  stg
boris@Boris:/usr/local/cpp_linux_labs$ git branch -r
origin/dev
origin/main
origin/prd
origin/stg
```

3.4. Создать скрипт автоматического переноса ревизий из ветки dev в ветку stg с установкой метки времени (tag). Скрипт в корень репозитория

Запросы:

```
boris@Boris:/usr/local/cpp_linux_labs$ nano \
merge_dev_to_stg.sh
boris@Boris:/usr/local/cpp_linux_labs$ chmod \
+x merge_dev_to_stg.sh
boris@Boris:/usr/local/cpp_linux_labs$ git add \
merge_dev_to_stg.sh
boris@Boris:/usr/local/cpp_linux_labs$ git commit \
-m "adding merge script"
```

```
" Inside merge_dev_to_stg.sh "
#!/bin/bash
git checkout stg
git merge dev --no-edit
TAG_NAME="release-$(date +%Y%m%d-%H%M%S)"
git tag "$TAG_NAME"
git push origin stg
git push origin "$TAG_NAME"
git checkout dev
```

Результат:

```
boris@Boris:/usr/local/cpp_linux_labs$ ./merge_dev_to_stg.sh
Switched to branch 'stg'
Your branch is up to date with 'origin/stg'.
Updating a556adc..1ecb771
Fast-forward
 .gitignore          | 2 ++
 merge_dev_to_stg.sh | 8 +++++++
 merge_stg_to_prd.sh | 8 +++++++
 3 files changed, 18 insertions(+)
 create mode 100644 .gitignore
 create mode 100755 merge_dev_to_stg.sh
 create mode 100755 merge_stg_to_prd.sh
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/sevsvs/cpp_linux_labs.git
   a556adc..1ecb771  stg -> stg
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/sevsvs/cpp_linux_labs.git
 * [new tag]         \
   release-20260404-002424 -> release-20260404-002424
Switched to branch 'dev'
Your branch is up to date with 'origin/dev'.
```

3.5. Создать скрипт автоматического переноса ревизий из ветки stg в ветку prd с установкой метки времени (tag). Скрипт в корень репозитория

Запросы:

```
boris@Boris:/usr/local/cpp_linux_labs$ nano \
merge_stg_to_prd.sh
boris@Boris:/usr/local/cpp_linux_labs$ chmod \
+x merge_stg_to_prd.sh
boris@Boris:/usr/local/cpp_linux_labs$ git add \
merge_stg_to_prd.sh
boris@Boris:/usr/local/cpp_linux_labs$ git commit \
-m "adding merge script"
```

```
" Inside merge_stg_to_prd.sh "  
#!/bin/bash  
git checkout prd  
git merge stg --no-edit  
TAG_NAME="production-$(date +%Y%m%d-%H%M%S)"  
git tag "$TAG_NAME"  
git push origin prd  
git push origin "$TAG_NAME"  
git checkout dev
```

Результат:

```
boris@Boris:/usr/local/cpp_linux_labs$ ./merge_stg_to_prd.sh  
Switched to branch 'prd'  
Your branch is up to date with 'origin/prd'.  
Updating a556adc..1ecb771  
Fast-forward  
 .gitignore          | 2 ++  
merge_dev_to_stg.sh | 8 ++++++++  
merge_stg_to_prd.sh | 8 ++++++++  
3 files changed, 18 insertions(+)  
create mode 100644 .gitignore  
create mode 100755 merge_dev_to_stg.sh  
create mode 100755 merge_stg_to_prd.sh  
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0  
To https://github.com/sevsvs/cpp_linux_labs.git  
 a556adc..1ecb771 prd -> prd  
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0  
To https://github.com/sevsvs/cpp_linux_labs.git  
 * [new tag]    \  
 production-20260404-002537 -> production-20260404-002537  
Switched to branch 'dev'  
Your branch is up to date with 'origin/dev'.
```

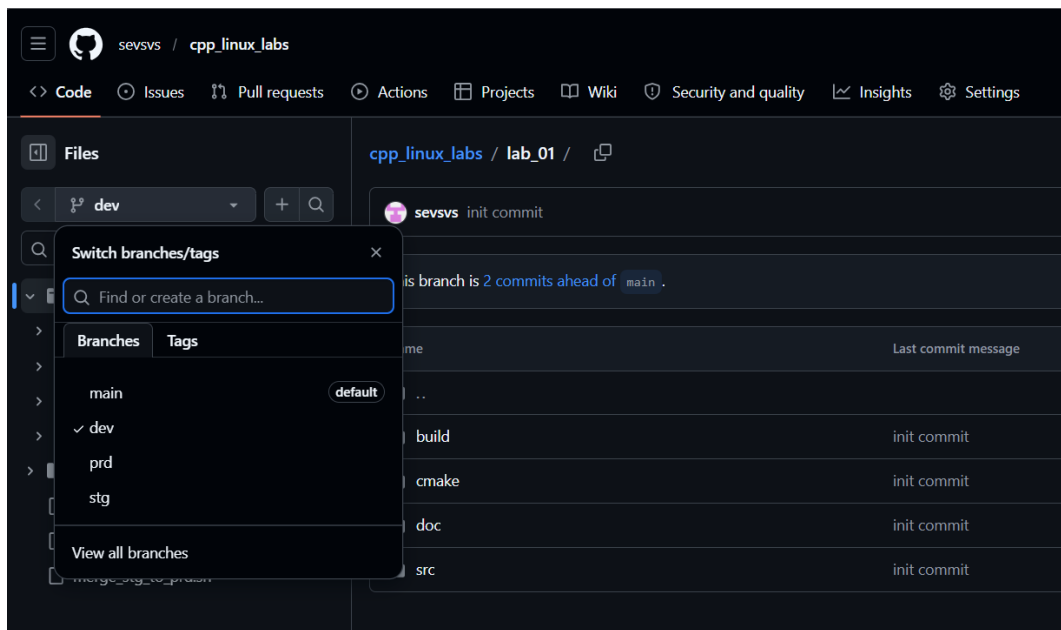


Рис. 1: Резултат на Github